# VBCUA: Voice-Based Concept Understanding Analyser

## Project Description:

VBCUA (Voice-Based Concept Understanding Analyser) harnesses speech recognition and natural language understanding to automatically assess how well a person can verbally explain a technical concept. The platform records or accepts an uploaded audio explanation, transcribes it in real time using OpenAI Whisper, compares the meaning of the transcript against a reference concept definition using Sentence-BERT semantic similarity, and analyses speaking fluency (pauses, filler words, pitch and energy) using Librosa. The two dimensions are combined into a single weighted score with automatically generated written feedback.

The system uses a modular architecture: a browser-based single-page frontend, a lightweight Node/Express server that serves the UI and proxies API calls, and a Python FastAPI backend that runs the AI pipeline (Whisper transcription, SBERT scoring, Librosa audio analysis, and ReportLab PDF report generation). This separation keeps the AI-heavy Python workload isolated from the web-serving layer, and lets VBCUA generate an instant score, interactive charts, and a downloadable PDF report for every submission.

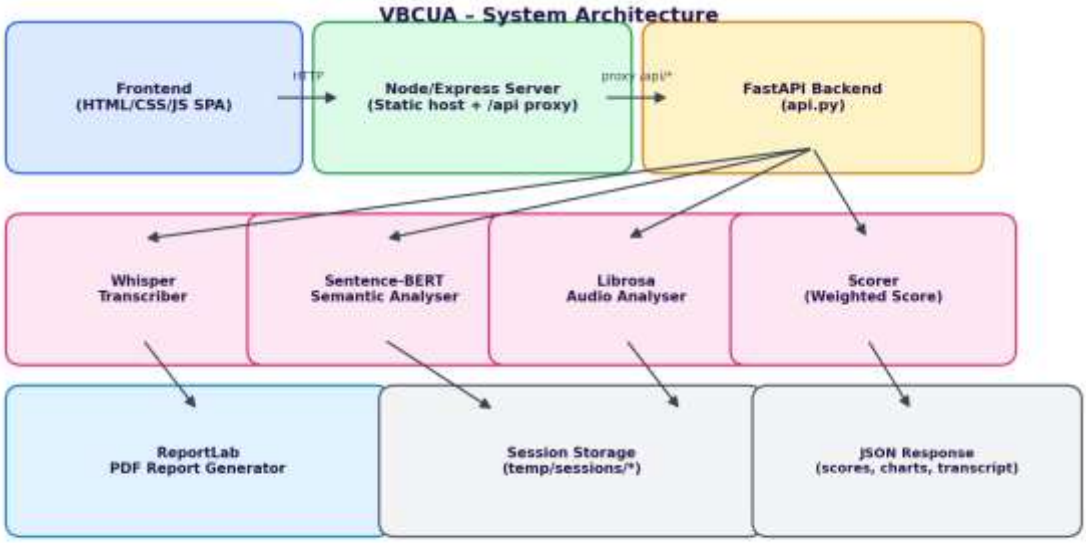
### Scenarios

**Scenario 1:** A student selects the "Machine Learning" concept and records a clear, well-structured explanation covering supervised/unsupervised learning and common algorithms. VBCUA transcribes the audio, finds high semantic similarity to the reference definition, detects minimal filler words and a steady speaking rate, and returns a "Strong Understanding" label with a green score badge and encouraging feedback.

**Scenario 2:** A student explains "Cybersecurity" but the response is thin on specifics and includes several hesitations ("um", "like", long pauses). The semantic score is decent but fluency drags the weighted score down into the "Moderate Understanding" band. The generated feedback specifically calls out the filler-word count and recommends practising a steadier pace.

**Scenario 3:** A student picks "Blockchain" but the explanation drifts off-topic and misses most of the key terms in the reference definition. The keyword gap analysis flags the missing concepts, the semantic similarity score is low, and VBCUA returns a "Poor Understanding" label along with specific study suggestions in the feedback paragraph and the downloadable PDF report.

## Technical Architecture:




**Description:** VBCUA uses a three-tier architecture. The frontend is a single-page HTML/CSS/JavaScript application that handles audio upload/recording and renders results. A Node/Express server serves this frontend and transparently proxies all /api/* calls (with an extended timeout for long AI processing) to the Python FastAPI backend running on port 8000. The FastAPI backend orchestrates the AI pipeline — Whisper for transcription, Sentence-BERT for semantic similarity, and Librosa for fluency metrics — combines the results in a weighted scorer, and uses ReportLab to generate a downloadable PDF report before returning a structured JSON payload to the frontend.

**Note:** VBCUA does not use a database — session audio and generated PDF reports are currently stored as local files under temp/sessions/{session_id}/, keyed by a UUID generated per analysis request (see the Scalability & Future Plan for the proposed move to object storage + PostgreSQL).

### Pre-requisites:

- **Python Programming Proficiency:** Core language for the FastAPI backend and AI modules.
- **FastAPI Framework Knowledge:** Async routes, file uploads, CORS middleware.
- **Speech & NLP Libraries:** OpenAI Whisper, Sentence-Transformers (SBERT), Librosa.
- **Node.js / Express Familiarity:** Static file serving and API proxying (http-proxy-middleware).
- **HTML, CSS, and JavaScript Skills:** Building the single-page frontend and Fetch-based API calls.
- **ReportLab / Matplotlib:** Generating PDF reports with embedded waveform and spectrogram charts.
- **Version Control with Git:** Tracking changes across backend, frontend, and server code.
- **Virtual Environment Setup:** Isolating Python dependencies (venv + requirements.txt).

## Project Workflow:

### Milestone 1: Requirement Analysis & System Architecture

- **Activity 1.1:** Define the problem statement — manually grading spoken concept explanations is slow, subjective, and doesn't scale — and set VBCUA's objective of automating it.
- **Activity 1.2:** Research and select the AI models: Whisper 'base' for transcription and 'all-MiniLM-L6-v2' (Sentence-BERT) for semantic similarity, balancing accuracy against local CPU inference speed.
- **Activity 1.3:** Define the three-tier architecture detailing interactions between the frontend, the Node/Express proxy, and the FastAPI backend's AI modules.
- **Activity 1.4:** Set up the development environment — a Python virtual environment for the backend and an npm project for the Node server and frontend tooling.

### Milestone 2: Core AI Pipeline Development

- **Activity 2.1: Speech-to-Text Transcription —** Load and cache the Whisper model, resample audio to 16kHz mono with Librosa, and transcribe using OpenAI Whisper.

```
def transcribe_audio(audio_path, progress_callback=None) -> dict:
    model = _get_model()  # lazy-loaded Whisper('base')
    audio_data, sr = librosa.load(str(audio_path), sr=16_000, mono=True)
    duration = float(len(audio_data)) / sr
    result = model.transcribe(audio_data, verbose=False, word_timestamps=False)
    return {
        "text": result.get("text", "").strip(),
        "language": result.get("language", "unknown"),
        "duration": duration,
        "segments": result.get("segments", []),
    }
```

- **Activity 2.2: Semantic Similarity Scoring —** Encode the transcript and the reference concept with Sentence-BERT and compute cosine similarity; also run a keyword gap analysis.

```
def compute_similarity(transcript: str, reference: str) -> float:
    model = _get_sbert()  # SentenceTransformer('all-MiniLM-L6-v2')
    embs = model.encode([transcript, reference], convert_to_numpy=True)
    return max(0.0, min(1.0, _cosine_similarity(embs[0], embs[1])))

def identify_gaps(transcript: str, reference: str) -> dict:
    keywords = identify_key_concepts(reference)   # top-20 heuristic keywords
    transcript_lower = transcript.lower()
    return {kw: (kw in transcript_lower) for kw in keywords}
```

- **Activity 2.3: Audio Fluency Analysis —** Use Librosa to detect pauses/silence, count filler words from a curated list, and compute pitch, RMS energy, and speaking rate (WPM).
- **Activity 2.4: Weighted Scoring & Feedback —** Combine semantic similarity (60%) and a fluency sub-score (25% energy + 30% pause ratio + 25% filler density + 20% speaking rate, 40% overall) into one final score with an auto-generated feedback paragraph.

```
final = (WEIGHT_SEMANTIC * semantic_similarity) + (WEIGHT_FLUENCY * fluency_score)
# WEIGHT_SEMANTIC = 0.60, WEIGHT_FLUENCY = 0.40
if final >= SCORE_STRONG:      label = "Strong Understanding"   # >= 0.75
elif final >= SCORE_MODERATE:  label = "Moderate Understanding" # >= 0.50
else:                          label = "Poor Understanding"
```

### Milestone 3: Backend API Development (FastAPI)

- **Activity 3.1:** Initialise the FastAPI app with permissive CORS middleware so the Node proxy and browser can call it during development.
- **Activity 3.2:** Implement POST /api/analyse — accepts an audio file, a concept name, and options; runs the full pipeline (transcribe → analyse audio → semantic scoring → weighted score → PDF generation) and returns one JSON payload.

```
@app.post("/api/analyse")
async def analyse(background_tasks: BackgroundTasks, audio: UploadFile = File(...),
                  concept: str = Form(...), options: str = Form(default="{}")):
    if concept not in CONCEPTS:
        raise HTTPException(400, f"Unknown concept '{concept}'.")
    session_id = str(uuid.uuid4())
    ...
    trans_result = transcribe_audio(audio_path)
    audio_result = analyse_audio(audio_path, transcript, duration)
    semantic_sim = compute_similarity(transcript, reference)
    score_result = compute_final_score(semantic_similarity=semantic_sim, ...)
    pdf_bytes = generate_pdf_report(concept=concept, transcript=transcript, ...)
    return JSONResponse(_numpy_safe(payload))
```

- **Activity 3.3:** Implement GET /api/report/{session_id} to stream the generated PDF back for download, and GET /api/concepts / GET /api/health for the concept list and a health check.
- **Activity 3.4:** Add a _numpy_safe() helper that recursively converts NumPy types to native Python so scores and arrays serialise cleanly to JSON.

### Milestone 4: Frontend Development

- **Activity 4.1:** Build a single-page app (index.html) with a sidebar-navigated layout across four pages: Home, Analyse, Results, and About, with a light/dark theme toggle.
- **Activity 4.2:** On the Analyse page, build a drag-and-drop upload zone (with a 'Use Sample Audio' shortcut), a concept dropdown, and checkboxes for optional outputs (spectrogram, sentence-level similarity, gap analysis, PDF).
- **Activity 4.3:** Send the recording via the Fetch API as multipart form data to /api/analyse, and show a step-by-step processing panel (Transcription → Audio Analysis → Semantic Scoring → Scoring → PDF Generation) with a live progress bar.
- **Activity 4.4:** On the Results page, render the waveform, radar, and donut charts, the transcript, the keyword gap analysis, and a button to download the PDF report.

### Milestone 5: PDF Reporting & Node Proxy Layer

- **Activity 5.1:** Generate waveform and mel-spectrogram plots with Matplotlib and encode them to base64 PNGs for both the frontend charts and the PDF report.
- **Activity 5.2:** Assemble the PDF with ReportLab, embedding the transcript, scores, fluency metrics, gap analysis, and the waveform/spectrogram images.
- **Activity 5.3:** Build the Node/Express server that serves the static frontend and proxies every /api/* request to the FastAPI backend, with a 5-minute proxy timeout to accommodate longer Whisper inference.

```
app.use(createProxyMiddleware({
  target: PYTHON_API,          // http://localhost:8000
  changeOrigin: true,
  pathFilter: "/api",
  proxyTimeout: 300_000,        // 5 min for long AI processing
  timeout: 300_000,
}));
app.use(express.static(FRONTEND_DIR));
app.get("*", (req, res) => res.sendFile(path.join(FRONTEND_DIR, "index.html")));
```

### Milestone 6: Testing, Deployment & Demo

- **Activity 6.1:** Run the FastAPI backend locally (uvicorn on port 8000) and the Node server (port 3000), and verify the full pipeline end-to-end with the bundled sample.wav.
- **Activity 6.2:** Test across all 10 predefined concepts with varying audio quality, filler-word density, and pacing to confirm the Strong/Moderate/Poor labels behave as expected.
- **Activity 6.3:** Rehearse the live demo flow with the team, verifying the frontend chart rendering stays in sync with backend API response timing (see Team Involvement in Demonstration for the resolution applied here).
- **Activity 6.4:** Wrap up with a project conclusion summarising outcomes and the scalability roadmap for future phases.

## Exploring the Website's Web Pages:

### Home Page:

Description: The landing page introduces VBCUA with a hero section and call-to-action buttons, a "How It Works" explainer, and a grid of the ten available reference concepts users can be assessed on.

### Analyse Page:

Description: A glass-styled panel where users drag-and-drop or record an audio explanation (or use the bundled sample), pick a concept from the dropdown, toggle optional outputs (spectrogram, sentence-level similarity, gap analysis, PDF), and submit. A processing panel with a live progress bar and step indicators (Transcription, Audio Analysis, Semantic Scoring, Scoring, PDF Generation) is shown while the backend pipeline runs.

### Results Page:

Description: Displays the final weighted score with a coloured label (Strong / Moderate / Poor Understanding), the semantic and fluency sub-scores, auto-generated written feedback, the full transcript, waveform and spectrogram visualisations, keyword gap analysis, and a button to download the full PDF report.

### About Page:

Description: Explains the technology behind VBCUA (Whisper, Sentence-BERT, Librosa) and the scoring methodology, giving users context for how their explanation is being evaluated.

## Conclusion:

VBCUA is a voice-based assessment platform that combines OpenAI Whisper transcription, Sentence-BERT semantic similarity, and Librosa fluency analysis into a single weighted comprehension score with auto-generated feedback and a downloadable PDF report. Built with a FastAPI backend, a Node/Express proxy layer, and a vanilla HTML/CSS/JS frontend, the project demonstrates how targeted speech and NLP models can automate a task — evaluating spoken concept explanations — that is otherwise slow and subjective to grade manually. Looking ahead, the team's scalability plan calls for an async task queue, cloud storage and a database for sessions, GPU-backed inference, and support for custom concepts and multiple languages, positioning VBCUA to grow from a ten-concept prototype into a general-purpose spoken-understanding assessment tool for classrooms and training programmes.